

ROS2

DAY 2

ROS2 PUBLISHERS AND SUBSCRIBERS (PYTHON)

Complete Beginner-Friendly

Introduction

Day 2 focuses on **inter-node communication in ROS2** using **topics**.
By the end of this manual, you will be able to:

- Create a workspace and package in Python
 - Create a **publisher node** manually
 - Create a **subscriber node** manually
 - Understand **timers, callbacks, and QoS**
 - Build and run ROS2 nodes
-

1. Workspace Creation

First, create a workspace for your ROS2 projects:

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
```

- `mkdir -p ~/ros2_ws/src` → creates a workspace folder `ros2_ws` with subfolder `src`
 - `cd ~/ros2_ws` → move into workspace
-

2. Package Creation

Inside the `src` folder, create a Python ROS2 package:

```
cd ~/ros2_ws/src
ros2 pkg create my_py_pkg --build-type ament_python
```

This generates:

```
my_py_pkg/
├── package.xml
├── setup.py
├── setup.cfg
└── my_py_pkg/
    └── __init__.py
```

3. Publisher Node Creation

Manually create a Python file for the publisher:

```
cd ~/ros2_ws/src/my_py_pkg/my_py_pkg
touch publisher_node.py
chmod +x publisher_node.py
```

Publisher Node Code

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class Talker(Node):
    def __init__(self):
        super().__init__('talker') # Node name
        self.publisher_ = self.create_publisher(String, 'chatter', 10) #
Topic: /chatter, queue size 10
        self.timer = self.create_timer(1.0, self.timer_callback) # Call
every 1 second
        self.count = 0

    def timer_callback(self):
        msg = String()
        msg.data = f"Hello ROS2 {self.count}" # Message content
        self.publisher_.publish(msg) # Publish to topic
        self.get_logger().info(msg.data) # Print to console
        self.count += 1

def main(args=None):
    rclpy.init(args=args) # Initialize ROS2 Python client
    node = Talker() # Create node instance
    rclpy.spin(node) # Keep node running
    node.destroy_node() # Cleanup after shutdown
    rclpy.shutdown() # Shutdown ROS2 client

if __name__ == '__main__':
    main()
```

Explanation:

- `Node('talker')` → Node named talker
 - `create_publisher(String, 'chatter', 10)` → Publishes String messages to /chatter with queue size 10
 - `create_timer(1.0, timer_callback)` → Calls timer_callback every 1 second
 - `timer_callback` → Creates a message, publishes it, logs it
 - `rclpy.spin(node)` → Keeps the node alive
-

Register Publisher in setup.py

Open `setup.py` and add the publisher entry:

```
entry_points={
    'console_scripts': [
        'publisher_node = my_py_pkg.publisher_node:main',
    ],
}
```

4. Subscriber Node Creation

Manually create a Python file for the subscriber:

```
touch subscriber_node.py
chmod +x subscriber_node.py
```

Subscriber Node Code

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class Listener(Node):
    def __init__(self):
        super().__init__('listener') # Node name
        self.subscription = self.create_subscription(
            String,
            'chatter',          # Topic name
            self.listener_callback, # Callback function
            10                  # Queue size
        )
        self.subscription # Prevent unused variable warning

    def listener_callback(self, msg):
        self.get_logger().info(f"I heard: {msg.data}") # Log received
        message
```

```
def main(args=None):
    rclpy.init(args=args)
    node = Listener()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Explanation:

- Subscribes to /chatter topic
- Calls listener_callback whenever a message arrives
- Logs messages received

Register Subscriber in setup.py

```
entry_points={
    'console_scripts': [
        'subscriber_node = my_py_pkg.subscriber_node:main',
    ],
}
```

5. Build Workspace

```
cd ~/ros2_ws
colcon build
source install/setup.bash
```

6. Run Nodes

Run Publisher

```
ros2 run my_py_pkg publisher_node
```

Run Subscriber

Open another terminal:

```
ros2 run my_py_pkg subscriber_node
```

Expected Output:

```
[INFO] [listener]: I heard: Hello ROS2 0  
[INFO] [listener]: I heard: Hello ROS2 1  
...
```

7. Linux Commands Used

- `mkdir` → create folder
 - `cd` → change directory
 - `touch` → create empty file
 - `chmod +x` → make file executable
 - `ros2 pkg create` → create ROS2 package
 - `colcon build` → build workspace
 - `source install/setup.bash` → source workspace environment
 - `ros2 run` → run ROS2 nodes
-

8. QoS Explanation

- The last parameter in `create_publisher` or `create_subscription` (e.g., 10) is the **queue size / depth**
 - Depth defines **how many messages are stored if subscriber is slow**
 - Helps prevent memory overflow and ensures subscriber receives recent messages
-